

Banking System mini project

```
import java.util.*;
// Step 1 : Bank Account Entity (SRP)
class BankAccount {
    private int accountNumber;
    private String accountHolderName;
    private String accountType;
    private double balance;
    public BankAccount(int accountNumber,
                       String accountHolderName,
                       String accountType,
                       double balance) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.accountType = accountType;
        this.balance = balance;
    }
    public int getAccountNumber() {
        return accountNumber;
    }
    public String getAccountHolderName() {
        return accountHolderName;
    }
    public String getAccountType() {
        return accountType;
    }
    public double getBalance() {
        return balance;
    }
    public void setBalance(double balance) {
        this.balance = balance;
    }
}
// Step 2 : Bank Service Interface (OCP)

interface BankService {
    void deposit(BankAccount account,
                 double amount);
    void withdraw(BankAccount account,
                  double amount);
    void transfer(BankAccount sender,
                  BankAccount receiver,
                  double amount);
}

// Bank Service Implementation
class BankServiceImpl implements BankService {
    @Override
    public void deposit(BankAccount account,
                       double amount) {
        account.setBalance(
            account.getBalance() + amount);
        System.out.println("\nAmount Deposited Successfully");
        System.out.println("Current Balance : "
            + account.getBalance());
    }
    @Override
    public void withdraw(BankAccount account,
                        double amount) {
        if (amount <= account.getBalance()) {
            account.setBalance(
                account.getBalance() - amount);
            System.out.println("\nAmount Withdrawn Successfully");
            System.out.println("Current Balance : "
                + account.getBalance());
        } else {
            System.out.println("\nInsufficient Balance");
        }
    }
    @Override
    public void transfer(BankAccount sender,
                        BankAccount receiver,
                        double amount) {
        if (amount <= sender.getBalance()) {
```

```

        sender.setBalance(
            sender.getBalance() - amount);
        receiver.setBalance(
            receiver.getBalance() + amount);
        System.out.println("\nTransfer Successful");
        System.out.println("Sender Balance : "
            + sender.getBalance());
        System.out.println("Receiver Balance : "
            + receiver.getBalance());
    } else {
        System.out.println("\nTransfer Failed");
        System.out.println("Insufficient Balance");
    }
}

// Step 3 : Notification Interface
interface NotificationService {
    void sendNotification(BankAccount account,
        String message);
}

// Email Notification
class EmailNotification
    implements NotificationService {
    @Override
    public void sendNotification(BankAccount account,
        String message) {
        System.out.println("\n===== EMAIL =====");
        System.out.println("Account : "
            + account.getAccountHolderName());
        System.out.println("Message : "
            + message);
        System.out.println("=====");
    }
}

// SMS Notification
class SMSNotification
    implements NotificationService {
    @Override
    public void sendNotification(BankAccount account,
        String message) {
        System.out.println("\n===== SMS =====");
        System.out.println("Account : "
            + account.getAccountHolderName());
        System.out.println("Message : "
            + message);
        System.out.println("=====");
    }
}

//Step 3 (Extension)
//Add New Notification
//WhatsApp Notification
class WhatsAppNotification
    implements NotificationService {
    @Override
    public void sendNotification(BankAccount account,
        String message) {
        System.out.println("\n===== WHATSAPP =====");
        System.out.println("Account : "
            + account.getAccountHolderName());
        System.out.println("Message : "
            + message);
        System.out.println("=====");
    }
}

//Step 5
//Payment Method Interface
interface PaymentMethod {

```

```

        void makePayment(BankAccount account,
                        double amount);
    }

    //RTGS Payment
    class RTGSPayment
        implements PaymentMethod {
    @Override
    public void makePayment(BankAccount account,
                        double amount) {
        if (amount <= account.getBalance()) {
            account.setBalance(
                account.getBalance() - amount);
            System.out.println("\nRTGS Payment Successful");
            System.out.println("Amount : "
                + amount);
            System.out.println("Balance : "
                + account.getBalance());
        } else {
            System.out.println("\nInsufficient Balance");
        }
    }
}

//Add New Payment Method
//NEFT

class NEFTPayment
    implements PaymentMethod {
    @Override
    public void makePayment(BankAccount account,
                        double amount) {
        if (amount <= account.getBalance()) {
            account.setBalance(
                account.getBalance() - amount);
            System.out.println("\nNEFT Payment Successful");
            System.out.println("Amount : "
                + amount);
            System.out.println("Balance : "
                + account.getBalance());
        } else {
            System.out.println("\nInsufficient Balance");
        }
    }
}

//Add New Payment Method
//UPI
class UPIPayment
    implements PaymentMethod {
    @Override
    public void makePayment(BankAccount account,
                        double amount) {
        if (amount <= account.getBalance()) {
            account.setBalance(
                account.getBalance() - amount);
            System.out.println("\nUPI Payment Successful");
            System.out.println("Amount : "
                + amount);
            System.out.println("Balance : "
                + account.getBalance());
        } else {
            System.out.println("\nInsufficient Balance");
        }
    }
}

//Step 4 : Receipt Service (DIP)
class ReceiptService {
    private NotificationService notification;
    public ReceiptService(NotificationService notification) {
        this.notification = notification;
    }
    public void printReceipt(BankAccount account,

```

```

        String transaction,
        double amount) {
System.out.println("\n===== RECEIPT =====");
System.out.println("Account Number : "
    + account.getAccountNumber());
System.out.println("Account Holder : "
    + account.getAccountHolderName());
System.out.println("Account Type : "
    + account.getAccountType());
System.out.println("Transaction : "
    + transaction);
System.out.println("Amount : "
    + amount);
System.out.println("Balance : "
    + account.getBalance());
System.out.println("=====");
notification.sendNotification(
    account,
    transaction + " Successful");
}
}
//Main Class
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Account Number : ");
        int accountNo = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Account Holder Name : ");
        String holder = sc.nextLine();
        System.out.print("Enter Account Type : ");
        String type = sc.nextLine();
        System.out.print("Enter Opening Balance : ");
        double balance = sc.nextDouble();
        BankAccount account =
            new BankAccount(accountNo,
                holder,
                type,
                balance);
        BankService bankService =
            new BankServiceImpl();
        NotificationService email =
            new EmailNotification();
        NotificationService sms =
            new SMSNotification();
        NotificationService whatsapp =
            new WhatsAppNotification();
        ReceiptService receipt =
            new ReceiptService(email);
        int choice;
        do {
            System.out.println("\n===== BANK MENU =====");
            System.out.println("1. Deposit");
            System.out.println("2. Withdraw");
            System.out.println("3. Transfer");
            System.out.println("4. RTGS Payment");
            System.out.println("5. NEFT Payment");
            System.out.println("6. UPI Payment");
            System.out.println("7. Email Notification");
            System.out.println("8. SMS Notification");
            System.out.println("9. WhatsApp Notification");
            System.out.println("10. Print Receipt");
            System.out.println("11. Exit");
            System.out.print("Enter Choice : ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.print("Enter Deposit Amount : ");
                    double deposit = sc.nextDouble();
                    bankService.deposit(account,
                        deposit);
                    receipt = new ReceiptService(email);
                    receipt.printReceipt(account,
                        "Deposit",
                        deposit);

```

```

        break;
case 2:
    System.out.print("Enter Withdraw Amount : ");
    double withdraw = sc.nextDouble();
    bankService.withdraw(account,
        withdraw);
    receipt = new ReceiptService(sms);
    receipt.printReceipt(account,
        "Withdraw",
        withdraw);
    break;
case 3:
    System.out.print("Enter Receiver Account Number : ");
    int receiverNo = sc.nextInt();
    BankAccount receiver =
        new BankAccount(receiverNo,
            "Receiver",
            "Savings",
            10000);
    System.out.print("Enter Transfer Amount : ");
    double transfer = sc.nextDouble();
    bankService.transfer(account,
        receiver,
        transfer);
    receipt = new ReceiptService(whatsapp);
    receipt.printReceipt(account,
        "Transfer",
        transfer);
    break;
case 4:
    System.out.print("Enter RTGS Amount : ");
    double rtgsAmount = sc.nextDouble();
    PaymentMethod rtgs =
        new RTGSPayment();
    rtgs.makePayment(account,
        rtgsAmount);
    receipt = new ReceiptService(email);
    receipt.printReceipt(account,
        "RTGS Payment",
        rtgsAmount);
    break;
case 5:
    System.out.print("Enter NEFT Amount : ");
    double neftAmount = sc.nextDouble();
    PaymentMethod neft =
        new NEFTPayment();
    neft.makePayment(account,
        neftAmount);
    receipt = new ReceiptService(sms);
    receipt.printReceipt(account,
        "NEFT Payment",
        neftAmount);
    break;
case 6:
    System.out.print("Enter UPI Amount : ");
    double upiAmount = sc.nextDouble();
    PaymentMethod upi =
        new UPIPayment();
    upi.makePayment(account,
        upiAmount);
    receipt = new ReceiptService(
        whatsapp);
    receipt.printReceipt(account,
        "UPI Payment",
        upiAmount);
    break;
case 7:
    email.sendNotification(
        account,
        "Email Notification Sent Successfully");
    break;
case 8:
    sms.sendNotification(
        account,
        "SMS Notification Sent Successfully");
    break;
case 9:

```

```

        whatsapp.sendNotification(
            account,
            "WhatsApp Notification Sent Successfully");
        break;
    case 10:
        receipt = new ReceiptService(email);
        receipt.printReceipt(
            account,
            "Account Summary",
            0);
        break;
    case 11:
        System.out.println(
            "\nThank You...");
        break;
    default:
        System.out.println(
            "\nInvalid Choice...");
    }
} while (choice != 11);
sc.close();
}
}

```

UML Diagram

